

AI CV Evaluator

Study Case Submission

1. Title

AI CV Evaluator - RAG-Powered Screening Service for CVs and Projects.

2. Candidate Information

Full Name: Hilqudz Dzikri

Email Address: hilqudzdzikri.hd@gmail.com

3. Repository Link

GitHub Repository: <https://github.com/hilqudz/ai-cv-evaluator>

4. Approach & Design (Main Section)

Initial Plan

How you broke down the requirements:

- Analyzed requirements for API endpoints (upload, evaluate, result)
- Designed asynchronous processing with queue system
- Planned for PDF text extraction and AI analysis integration
- Structured database schema for uploads and evaluation jobs
- Implemented RAG enhancement with ChromaDB for better analysis

Key assumptions or scope boundaries:

- PDF files only (no other document formats)
- Maximum file size of 10MB to prevent resource exhaustion
- English language content for AI analysis
- Redis queue system for reliable background processing
- PostgreSQL database for robust data storage

System and Database Design

API Workflow:

Step	Endpoint	What Happens	Response
1	<code>POST /api/upload</code>	Upload PDF files, validate, store	Upload ID
2	<code>POST /api/evaluate</code>	Create job, queue for processing	Job ID
3	<code>GET /api/result/{id}</code>	Check job status and get results	Analysis results

Background Processing (What happens in the queue):

1. **Extract PDF text** using pdftotext or PHP fallback
2. **Get context from ChromaDB** (job matching + evaluation criteria)
3. **Analyze CV** using Gemini AI with enhanced prompts
4. **Analyze Project** using Gemini AI with project criteria
5. **Generate summary** and save results to database

Key Technologies:

- **Laravel 12:** API framework
- **PostgreSQL:** Main database (Supabase)
- **Redis:** Queue system for background jobs
- **Google Gemini:** AI analysis
- **ChromaDB:** Vector database for RAG enhancement

System & Database Design

API Endpoints Design:

- `POST /api/upload` - File upload endpoint for CVs and job descriptions
- `POST /api/evaluate` - Initiate CV evaluation process
- `GET /api/result/{id}` - Retrieve evaluation results

Database Schema:

uploads table:

```
CREATE TABLE uploads (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    original_filename VARCHAR(255) NOT NULL,  
    storage_path VARCHAR(255) NOT NULL, -- Path in storage/app/  
    file_type VARCHAR(255) NOT NULL, -- 'cv' or 'project_report'  
    file_size INTEGER NOT NULL, -- Size in bytes  
    mime_type VARCHAR(255) NOT NULL, -- 'application/pdf'  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP  
);
```

evaluation_jobs table:

```
CREATE TABLE evaluation_jobs (  
    id UUID PRIMARY KEY, -- UUID for job tracking  
    job_title VARCHAR(255) NOT NULL,  
    cv_upload_id BIGINT NULL,  
    project_report_upload_id BIGINT NULL,  
    status ENUM('queued', 'processing', 'completed', 'failed') DEFAULT 'queued',  
    result JSON NULL, -- Stores complete evaluation results  
    completed_at TIMESTAMP NULL,  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP,  
  
    FOREIGN KEY (cv_upload_id) REFERENCES uploads(id),  
    FOREIGN KEY (project_report_upload_id) REFERENCES uploads(id)  
);
```

Key Design Decisions:

- UUID primary key for evaluation_jobs enables secure job tracking without exposing sequential IDs
- JSON result field stores complete evaluation data with flexible schema
- Nullable foreign keys allow partial uploads (CV only or project only)
- Enum status field provides clear job state tracking

Job Queue / Long-running Task Handling:

- **Redis Queue System:** Production-ready message queue for asynchronous processing
- **ProcessEvaluationJob Configuration:**

Parameter	Value	Purpose
Maximum Tries	3 attempts	Automatic retry on transient failures
Job Timeout	300 seconds (5 minutes)	Prevents resource hoarding from hanging processes
Queue Connection	Redis	Persistent message storage with high availability

Failed Job Handling	Database status update	Comprehensive error logging with stack traces
---------------------	------------------------	---

- **Multi-stage Processing Pipeline:**
 1. PDF Text Extraction (with fallback mechanisms)
 2. ChromaDB Context Retrieval (with graceful degradation)
 3. AI Analysis (CV evaluation → Project evaluation → Final summary)
 4. Result Storage (JSON format in database)
 5. Status Tracking (queued → processing → completed/failed)
- **Queue Workers:** Background processes handle jobs asynchronously, enabling API to return immediately
- **Monitoring:** Real-time job status tracking via database with detailed error information

LLM Integration

Why you chose a specific LLM or provider:

- It's free and has good rate limits for testing
- Handles complex prompts well and gives consistent results
- Good at following structured output formats
- The API is reliable and well-documented

Prompt design decisions:

- Made them super structured so I could parse the responses with regex
- Used low temperature (0.1) so scores stay consistent
- Split CV and project analysis into separate prompts for clarity
- Added clear scoring criteria so the AI knows what to look for

Chaining logic:

- Three-stage analysis: CV evaluation → Project evaluation → Final summary
- Each stage feeds results to the next for comprehensive assessment
- RAG-enhanced prompts when ChromaDB data is available
- Fallback to standard prompts if RAG enhancement fails

RAG (retrieval, embeddings, vector DB) strategy:

- **ChromaDB Integration:** Production-ready vector database service
- **Database Configuration:** Tenant-based multi-tenancy with API key authentication
- **Four Collections with Specific Purposes:**

Collection Name	Purpose	Data Structure	Usage in Pipeline
<code>job_descriptions</code>	Job matching enhancement	Complete job requirements with metadata (title, company, year, focus area)	Vector similarity search to find matching job requirements for CV analysis
<code>cv_evaluation_rubrics</code>	Standardized CV scoring	Detailed scoring criteria (1-5 scale) with weights: Technical Skills (40%), Experience (25%), Achievements (20%), Cultural Fit (15%)	Retrieved during CV analysis to ensure consistent evaluation standards
<code>project_evaluation_rubrics</code>	Project assessment criteria	5-dimension scoring rubric: Correctness (30%), Code Quality (25%), Resilience (20%), Documentation (15%), Creativity (10%)	Used for project report evaluation with weighted scoring calculation
<code>case_study_brief</code>	Context for project evaluation	Complete case study requirements including technical specifications and deliverable expectations	Provides context for evaluating project alignment with original requirements

- **Vector Search Implementation:** Semantic similarity matching using cosine distance for context retrieval
- **Enhanced Prompt Generation:** Retrieved context dynamically injected into AI prompts for better evaluation accuracy
- **Graceful fallback:** Standard prompt analysis when vector database is unavailable

Prompting Strategy:

Prompting Strategy

Three-Stage Evaluation System:

1. CV Analysis Prompt:

You are an expert HR analyst evaluating a candidate's CV for a {jobTitle} position.

JOB DESCRIPTION:

{jobDescription}

CANDIDATE CV:

{cvContent}

Please analyze the CV and provide a structured evaluation with the following format:

MATCH_RATE: [0.0-1.0 decimal representing how well candidate matches job requirements]

TECHNICAL_SKILLS_SCORE: [1-5 score for technical skills alignment]

EXPERIENCE_SCORE: [1-5 score for experience level and relevance]

ACHIEVEMENTS_SCORE: [1-5 score for relevant achievements and impact]

CULTURAL_FIT_SCORE: [1-5 score for communication and collaboration indicators]

FEEDBACK:

[Detailed 2-3 sentences explaining strengths and gaps in candidate's background relative to job requirements]

2. Project Analysis Prompt:

You are an expert technical reviewer evaluating a candidate's project submission.

CASE STUDY REQUIREMENTS:

{caseStudyBrief}

CANDIDATE PROJECT REPORT:

{projectContent}

IMPORTANT: Respond ONLY in this exact format, with no additional text:

CORRECTNESS_SCORE: [1-5 score for technical accuracy and requirement fulfillment]

CODE_QUALITY_SCORE: [1-5 score for code structure, patterns, and best practices]

RESILIENCE_SCORE: [1-5 score for error handling and robustness]

DOCUMENTATION_SCORE: [1-5 score for clarity and completeness of documentation]

CREATIVITY_SCORE: [1-5 score for innovative approaches and problem-solving]

PROJECT_FEEDBACK:

[Detailed analysis of technical implementation, strengths, and areas for improvement]

3. Final Summary Prompt:

You are a senior hiring manager making a final recommendation for a {jobTitle} position.

CV EVALUATION SUMMARY:

- Match Rate: {cvScore}

- Feedback: {cvFeedback}

PROJECT EVALUATION SUMMARY:

- Overall Score: {projectScore}

- Feedback: {projectFeedback}

Please provide a final hiring recommendation:

OVERALL_SUMMARY:

[3-4 sentences with: 1) Key strengths, 2) Areas of concern, 3) Final recommendation (hire/interview/reject), 4) Next steps if applicable]

Resilience & Error Handling

How you handled API failures, timeouts, or randomness:

- 60-second timeout for Gemini API requests to prevent hanging
- Comprehensive error logging with context (prompt length, error details)
- Graceful degradation: RAG enhancement fails → fallback to standard prompts
- Job status tracking with detailed error information stored in database
- Low temperature (0.1) setting to minimize response randomness

Any retry, backoff, or fallback logic:

- Queue job retry mechanism: 3 attempts with automatic backoff
- 5-minute timeout per job execution to prevent resource hoarding
- Failed job handler updates database status with error details
- PDF extraction fallback: pdftotext binary → PHP-based extraction
- RAG system failure → standard prompt analysis without vector enhancement

Edge Cases Considered

What unusual inputs or scenarios you thought about:

- Large file sizes (10MB+ PDFs) - handled with file size validation
- Corrupted or password-protected files - graceful error handling with user feedback
- Non-text PDFs (scanned images) - fallback extraction with warning messages
- Empty or minimal CV content - validation and meaningful error responses
- Special characters and Unicode in file names - proper sanitization
- Concurrent upload requests - proper file handling and unique naming
- API rate limiting - retry logic with exponential backoff
- Network timeouts during long AI processing - robust timeout handling

How you tested them:

- Created comprehensive test suites with edge case scenarios
- Unit tests for PDF extraction service with various file types
- Feature tests for API endpoints with invalid inputs
- Mock testing for external service failures (Gemini API, ChromaDB)

- File upload tests with oversized and corrupted files
- Concurrent request testing for race condition detection

5. Results & Reflection

Outcome

What worked well in your implementation?

- **Asynchronous Processing:** Redis queue system handled background jobs efficiently, preventing API timeouts
- **Error Handling:** Comprehensive error handling with retry mechanisms ensured system reliability
- **PDF Extraction:** Dual-method approach (pdftotext + PHP fallback) achieved high success rate across different PDF types
- **RAG Integration:** ChromaDB vector database successfully enhanced AI analysis with contextual job matching
- **API Design:** RESTful endpoints with proper status codes and JSON responses for easy integration
- **Database Design:** UUID-based job tracking and JSON result storage provided flexibility and security

What didn't work as expected?

- **AI Response Consistency:** Occasional variance in Gemini API responses despite low temperature settings required additional response validation
- **Large File Processing:** PDF extraction for very large files (>5MB) sometimes exceeded memory limits on constrained environments
- **RAG Dependency:** ChromaDB service outages caused fallback to standard prompts, reducing analysis quality
- **Development Setup:** Initial ChromaDB configuration required more documentation for team onboarding

Evaluation of Results

Evaluation quality and consistency:

The evaluation results proved to be stable and meaningful due to several key factors:

- **Structured Prompts:** Regex-parseable output format ensured consistent data extraction
- **Multi-stage Analysis:** Separate CV and project evaluation provided comprehensive assessment
- **RAG Enhancement:** Vector database context improved accuracy by matching similar job requirements
- **Scoring Validation:** Input validation and range checks prevented invalid scores

- **Low Temperature Setting:** 0.1 temperature minimized randomness while maintaining analysis quality

The system successfully processed real CV and project data with consistent scoring patterns and actionable feedback.

Future Improvements

What would you do differently with more time?

- **Enhanced AI Pipeline:** Implement GPT-4 or Claude integration for comparison and ensemble scoring
- **Advanced RAG:** Add semantic chunking and re-ranking for more sophisticated context retrieval
- **Performance Optimization:** Implement caching layer for repeated analyses and optimize PDF processing
- **User Interface:** Build web dashboard for real-time job monitoring and result visualization
- **Analytics Dashboard:** Add metrics tracking for evaluation patterns and system performance

What constraints (time, tools, API limits) affected your solution?

- **API Rate Limits:** Gemini API free tier limited concurrent processing capacity
- **Development Time:** Limited time prevented implementation of advanced caching and optimization
- **Resource Constraints:** ChromaDB free tier restricted vector database size and operations
- **Testing Environment:** Limited access to diverse PDF samples for comprehensive edge case testing
- **Documentation Time:** Comprehensive API documentation took significant time away from feature development

6. Screenshots of Real Responses

File Upload Response

ai-cv-evaluator / upload cv

SaveShare

POSThttp://localhost:8000/api/uploadSend

DocsParamsAuthorizationHeaders (9)BodyScriptsSettingsCookies

noneform-datax-www-form-urlencodedrawbinaryGraphQL

	Key	Value	Description	Bulk Edit
☒	cv	FileHilqudz Dzikri CV Updated 2025.pdf		
☒	project_report	FileAI CV Evaluator - Laravel API with Goo...		
	Key	TextValue	Description	

BodyCookiesHeaders (7)Test Results201 Created1.58 s487 BSave Response

JSONPreviewVisualize

```
1 {
2   "success": true,
3   "message": "Files uploaded successfully",
4   "data": {
5     "cv": {
6       "id": 27,
7       "filename": "Hilqudz Dzikri CV Updated 2025.pdf",
8       "size": 193772
9     },
10    "project_report": {
11      "id": 28,
12      "filename": "AI CV Evaluator - Laravel API with Google Gemini Integration.pdf",
13      "size": 980780
14    }
15  }
16 }
```

Evaluation Initiation Response

ai-cv-evaluator / evaluate

SaveShare

POSThttp://localhost:8000/api/evaluateSend

DocsParamsAuthorizationHeaders (9)BodyScriptsSettingsCookies

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

SchemaBeautify

```
1 {
2   "cv_upload_id": 27,
3   "project_report_upload_id": 28,
4   "job_title": "Backend Developer"
5 }
6 }
```

BodyCookiesHeaders (7)Test Results201 Created2.82 s313 BSave Response

JSONPreviewVisualize

```
1 {
2   "success": true,
3   "data": {
4     "id": "3a41cbf0-38c2-4e89-92ac-fd1d831811db",
5     "status": "queued"
6   }
7 }
```

Final Evaluation Results

ai-cv-evaluator / get result

Save

GET

http://localhost:8000/api/result/3a41cbf0-38c2-4e80-92ac-fd1d831811db

Send

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description		Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (7)

Test Results

200 OK · 1.71 s · 1.74 KB

Save Response

JSON

Preview

Visualize

```
1 {
2   "id": "3a41cbf0-38c2-4e80-92ac-fd1d831811db",
3   "status": "completed",
4   "result": {
5     "cv_match_rate": 0.85,
6     "cv_feedback": "Hilqudz is a strong candidate with a solid foundation in backend development, particularly with PHP,
7                     Laravel Lumen, and Golang, aligning well with the job requirements. The CV highlights quantifiable achievements and
8                     demonstrates a collaborative mindset, making them a potentially excellent fit for the team. While the CV doesn't
9                     explicitly mention experience with specific database systems like MySQL or PostgreSQL, their experience with SQL and
10                    RESTful APIs suggests a transferable skillset.",
11     "project_score": 3.9,
12     "project_feedback": "The candidate delivered a functional AI-powered CV evaluation system with a well-defined
13                         architecture and integration with Google Gemini and ChromaDB. The asynchronous processing and RAG implementation are
14                         commendable. Documentation could be more detailed, and further resilience testing would be beneficial.",
15     "overall_summary": "Hilqudz presents a strong profile with their proficiency in PHP, Laravel Lumen, and Golang, coupled
16                        with a functional AI-powered CV evaluation project demonstrating practical application of relevant technologies.
17                        While database experience isn't explicitly stated, their SQL and RESTful API knowledge suggests a transferable
18                        skillset. I recommend moving forward with a final interview to assess their database experience and delve deeper into
19                        the project's resilience and documentation. This interview will help confirm their suitability for the role and
20                        address any remaining concerns."
21   }
22 }
```

Submission Date: November 17, 2025